

Использование аппроксимационных умножителей в цифровых фильтрах

Выпускная квалификационная работа бакалавра

Сажнев М. Е.

Научный руководитель: Бахурин С. А.

Московский физико-технический институт

МФТИ, 2026

- Умножитель — доминирующий блок по площади и энергопотреблению в ЦОС
- В адаптивных фильтрах работает непрерывно $\Rightarrow$ определяет энергобюджет
- Сигнал всегда зашумлён (квантование АЦП, шум канала) $\Rightarrow$ его точность ограничена уровнем шума
- Точность умножения избыточна: младшие биты результата тонут в шуме $\Rightarrow$ **приближённые вычисления** — погрешность ниже уровня шума даёт **экономия без потери качества**
- Базовый приём — кодирование Бута по основанию 4 (вдвое меньше частичных произведений)

Ключевой вопрос

Как применить метод с дополнительной перекодировкой (Zhu et al., Университет Бэйхан, 2025) в реалистичном сценарии адаптивного фильтра?

Цель работы: снизить энергопотребление умножителя Бута в адаптивном цифровом фильтре, применив методы перекодировки с добавлением нулевых кодов.

Задачи:

1. Адаптировать алгоритм перекодировки (Zhu et al., 2025) к параллельной архитектуре умножителя
2. Предложить архитектурное решение, реализующее преимущества перекодировки в сценарии адаптивного фильтра
3. Реализовать предложенную архитектуру на Verilog
4. Построить методику измерения площади и мощности на общедоступных инструментах (45 нм)
5. Оценить точность приближённых режимов (NMED, MRED, SNR) на статистике 10^6 пар

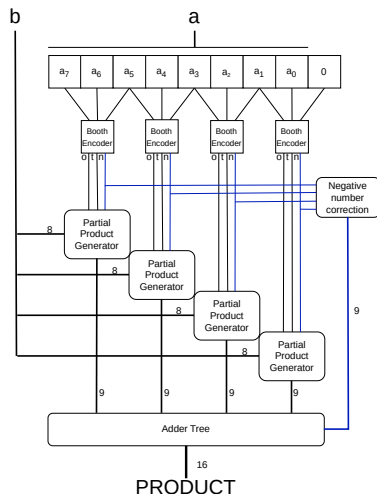
Умножитель Бута по основанию 4 в составе N -тапового адаптивного фильтра

Принцип Бута по основанию 4:

- Множитель — перекрывающиеся группы по 3 бита
- Каждая группа $\rightarrow$ код из набора $\{\bar{2}, \bar{1}, 0, 1, 2\}$ 3 бита: (One, Two, Neg)
- Вдвое меньше частичных произведений

Ключевая асимметрия

Коэффициенты обновляются **редко** (раз в $10-10^3$ тактов), поток отсчётов — **каждый такт**



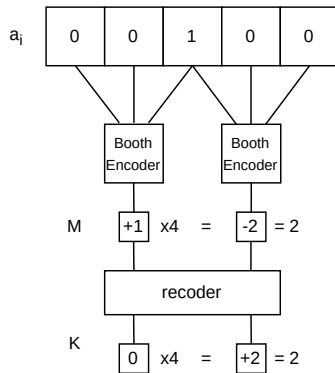
Идея (Zhu et al., Университет Бэйхан, 2025):
пара соседних ненулевых кодов Бута
переписывается в эквивалентную пару,
содержащую ноль:

$$K_{2i+1} = M_{2i+1} - x, \quad K_{2i} = M_{2i} + 4x$$

$$x = M_{2i+1} \Rightarrow K_{2i+1} = 0, \quad K_{2i} = M_{2i} + 4M_{2i+1}$$

- Точно (ER): паттерны 00100, 11011, ошибка ED = 0
- Приблизённо (AR): ещё 4 паттерна (00101, 00110, 11001, 11010), ED = ± 1

Больше нулевых кодов $\Rightarrow$ больше занулённых
частичных произведений.

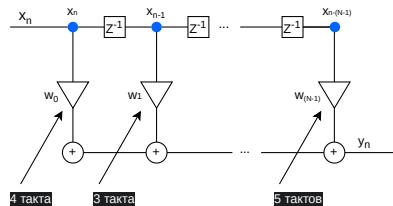


Проблема: последовательный умножитель в конвейере

В оригинале — последовательный умножитель:
нулевые коды **пропускаются**, число тактов
переменно.

КИХ-фильтр требует результата каждый такт $\Rightarrow$
нужен контроллер готовности:

- добавляет площадь и динамическую мощность
- накладные расходы перекрывают выигрыш
- реальный тракт ЦОС требует фиксированной пропускной способности



Вывод

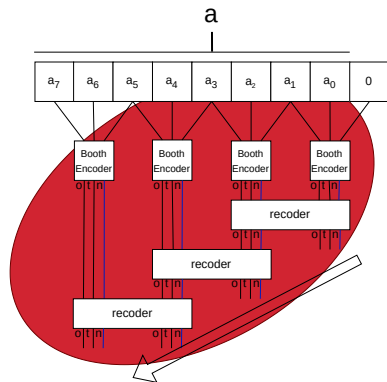
Необходим параллельный умножитель с
фиксированной одноктактной задержкой.

Проблема: перекодировка внутри умножителя

Алгоритм перекодировки обрабатывает пары кодов Бута **последовательно**, справа налево:

- Пары перекрываются $\Rightarrow$ каждая зависит от предыдущей $\Rightarrow$ нельзя распараллелить
- Перекодировщик — дополнительная комбинационная логика внутри умножителя $\Rightarrow$ сам потребляет энергию

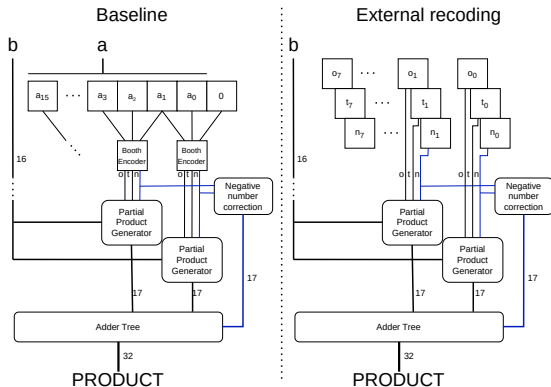
Простое добавление перекодировки в параллельный умножитель лишь ухудшает модуль.



Предлагаемая архитектура: вынесенная перекодировка

Ключевая идея: перекодировщик вынесен **наружу** умножителя.

- Один перекодировщик считает вектор для всех тапов $\Rightarrow$ стоимость амортизируется по N тапам
- Благодаря асимметрии перекодировка считается **один раз** при обновлении
- В регистре умножителя хранится уже перекодированный вектор (24 бита: two, one, neg)
- Умножитель — один и тот же RTL для всех режимов (точный / приближённые)



Слева — базовая схема, справа — с вынесенной перекодировкой

Два RTL-модуля (Verilog):

- `mult_booth` — исходный: Бут по основанию 4, перекодировщик внутри (эталон для сравнения)
- `mult_booth_extrec` — предлагаемый: принимает готовый 24-битный вектор, перекодировщика нет

Перекодировщик — на Python:

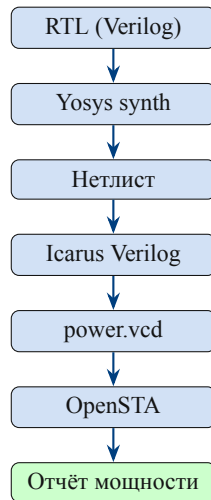
- последовательный алгоритм, обрабатывает все 7 пар кодов Бута
- генерирует вектор `{two, one, neg}` для умножителя
- `exact` — точный: нулевая ошибка, только 2 паттерна
- `approx_1..4` — приближённые: каждый следующий уровень добавляет паттерны с $ED = \pm 1$, агрессивность растёт

Маршрут синтеза и анализа:

1. Python recoder $\rightarrow$ stimulus-файлы
(a , b , recoded)
2. RTL $\xrightarrow{\text{Yosys}}$ нетлист (NanGate45, 45 нм)
3. Нетлист $\xrightarrow{\text{Icarus Verilog}}$ power.vcd
4. power.vcd $\xrightarrow{\text{OpenSTA}}$ отчёт мощности

Тестовые сценарии:

- Распределение: равномерное, гауссово ($\sigma=8000$)
- Период обновления a : fast ($T=1$), slow ($T=10$ тактов)
- $N=20\,000$ умножений;



Модуль	Площадь, мкм ²
mult_booth	2725
mult_booth_extrec	2698

Два компенсирующих фактора

— Из умножителя убрана вся логика кодирования Бута (вычисление `neg`, `one`, `two` из коэффициента)

+ Входной регистр коэффициента расширен: 16 бит → 24 бита (перекодированный вектор)

Уменьшение слегка перевешивает $\Rightarrow$ площадь не растёт (предлагаемый на 1% меньше)

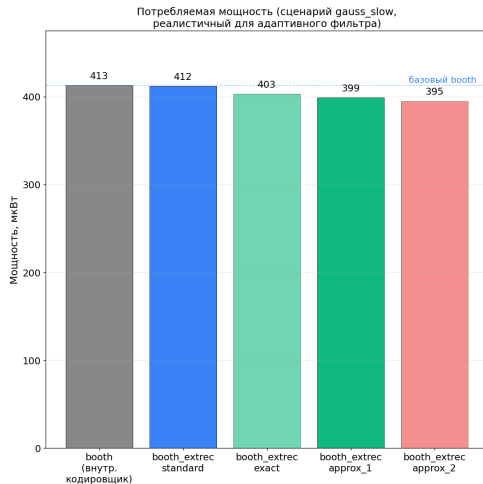
Сценарий gauss_slow (самый реалистичный):

Конфигурация	Мощность, мкВт	Экономия
booth (базовый)	413	—
extrec exact	403	+2.4%
extrec approx_1	399	+3.4%
extrec approx_2	395	+4.4%

Режим standard

Вынесенная архитектура, но с обычными коэффициентами, без перекодировки

Его мощность практически равна booth $\Rightarrow$ сам вынос логики мощность **не меняет**; всю экономию дают именно занулённые частичные произведения

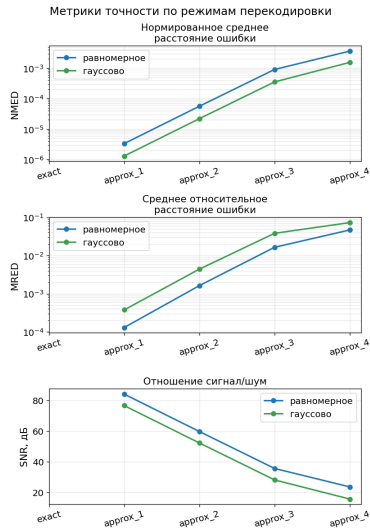


Режим	NMED	SNR, дБ
exact	0	∞
approx_1	$1.3 \cdot 10^{-6}$	76.7
approx_2	$2.2 \cdot 10^{-5}$	52.3
approx_3	$3.5 \cdot 10^{-4}$	28.1
approx_4	$1.5 \cdot 10^{-3}$	15.6

10^6 пар, гауссово распределение

Важное свойство

Систематическое смещение ≈ 0 :
 ED = +1 и -1 симметричны по
 конструкции



Режим	Экономия мощн.	SNR (гаусс)
exact	+2.4%	∞
approx_1	+3.4%	76.7 дБ
approx_2	+4.4%	52.3 дБ

Оценка approx_2 для практики

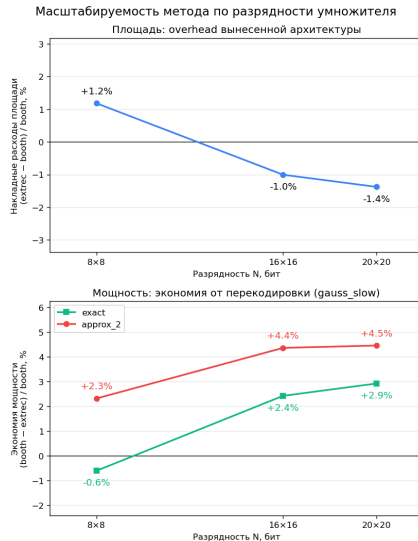
SNR = 52 дБ $\gg$ 40 дБ — безопасен для многих приложений ЦОС. Смещение ≈ 0 исключает накопление ошибки.

Масштабируемость по разрядности

Ожидание: с ростом разрядности всё бо́льшую часть умножителя занимает **суммирующее дерево** — именно на него и действует перекодировка. Значит, эффективность должна расти с разрядностью.

Подтверждено на 8/16/20 бит (единый маршрут синтеза):

- по площади архитектура умножитель практически не меняет (разница в пределах $\pm 1,5\%$)
- экономия мощности растёт с разрядностью:
 $+2,3 \rightarrow +4,4 \rightarrow +4,5\%$ (approx_2)



- Вынесенный кодировщик: -4.4% мощности (gauss_slow, approx_2)
- Площадь умножителя не возрастает (даже на 1% меньше) — два эффекта компенсируют друг друга
- Эффективность растёт с разрядностью (проверено 8/16/20 бит)
- Единый RTL для всех режимов точности
- approx_2: SNR = 52 дБ, смещение ≈ 0 — пригоден для большинства задач ЦОС

Главный результат

Перекодировщик вынесен наружу и усложнён методом дополнительной перекодировки: он создаёт нулевые частичные произведения до того, как данные попадают в умножитель. Сам умножитель при этом упрощается — его площадь не растёт, а мощность снижается.

Спасибо за внимание

Сажнев М. Е. МФТИ, 2026